

EVALUATION AND EXPERIMENTS :

All the experiments were conducted on a production HPC cluster : IBM LSF 9.1, comprised 22 active compute nodes (out of 32 total), running RHEL 6.4 (kernel 2.6.32) , the gateway was implemented as containerized microservice using docker compose , consisting of Nginx (TLS1,3 , reverse proxy), Keycloak 26 (OIDC identity provider), FastAPI 0.115 (Python 3.12 backend), and PostgreSQL 16

latency measurements were obtained using the python function (time.perf_counter) ,with 50 end to end trials and 100 component-level measurements

CARTA scenarios were evaluated using automated scripts that injected controlled attack patterns into the production system

table 1 represents latency measurements of end to end pipeline... the average total of latency is 7434 ms , with job submission (SFTP + SSH) dominating execution time at 5220.97 ms ...the security processing (auth+policy+carta) contribute 39 less than 1% than the total latency

Table 2 represents Security Layer Component Latency with JWT verification dominates security overhead at 38.70 ms, the system rejects malicious scripts in 26.49 ms (12.9×) faster than processing legitimate complex scripts (340.49 ms) -> this arises from early exit on the first forbidden token: malicious scripts typically import dangerous modules for ex import socket, eval ...in the first few lines

table 3 presents the evaluation of the CARTA risk engine across eight representative scenarios...

in the normal case (legitimate user clean history) : the risk score is 0.000 being admitted to submit the job , in the 6 attack scenarios the risk score values between 0.350 and 0.441 which triggers multi-factor authentication (MFA)

S7 : the session score decay confirms the EMA formula $0.441 \rightarrow 0.020$ after clean request

S8 : The cryptographic audit chain was verified across 280 production entries linked by 28 external HMAC anchor records (one anchor per 10 entries) , A single field modification (the field of entry) a record was injected directly via SQL to simulate a privileged database attacker. The verify_chain() endpoint immediately returned {valid:false}

JSON :

```
{
  "valid": true,
  "entries_checked": 280,
  "hmac_entries": 280,
  "tamper_resistant": true,
  "anchors_verified": 28,
  "message": "Full audit integrity verified"
```

```
}
```

```
// After single field modification via SQL:
```

```
{  
  "valid": false,  
  "broken_at": "61ccfd18-9f29-42fa-...",  
  "action": "job_submit",  
  "reason": "hash_mismatch — entry was modified"  
}
```